Chapter 2: Variables, Input and Output

Department of Mathematics, Faculty of Applied Science
King Mongkut's University of Technology North Bangkok

Course 040223101 – Computer Programming for Mathematics 1
Instructor: Assoc. Prof. Dr. Anuchit Jitpattanakul | Semester 1, Academic Year 2026

Learning Objectives

After completing this chapter, students will be able to:

01

Declare & Name Variables

Declare and assign values to variables following Python naming rules.

02

Identify Data Types

Classify the four basic data types: `int`, `float`, `str`, and `bool`.

03

Type Casting

Convert between data types using `int()`, `float()`, and `str()`.

04

Input & Output

Receive user input with `input()` and display results with `print()` and f-strings.

05

Math Operators & Formatting

Apply arithmetic operators and format output using f-string syntax.

This chapter aligns with Course Learning Outcome **CLO 2**.

Chapter Overview

A useful program must be able to **store data**, **receive input** from users, and **display results**. This chapter covers:

Variables

Like boxes that hold data in memory

Data Types

The kind of value each variable stores

Type Casting

Converting between data types

I/O Operations

The heart of the IPO model from Chapter 1

2.1 Variables and Assignment

What is a Variable?

A variable is a **name that refers to data stored in memory**. Assignment uses the `=` sign – it means "store the right-hand value into the left-hand variable," not mathematical equality.

Syntax: `variable_name = value`

File: 02variable.py

```
x = 25.2
name = "Somchai"
age = 19

print(x)
print(name)
print(age)
```

Output:

```
25.2
Somchai
19
```

2.2 Python Variable Naming Rules

✅ Allowed Characters

Letters A - Z, a - z, digits 0 - 9, and underscore `_` only.

❌ No Leading Digit

`1score` is invalid; `score1` is valid.

❌ No Spaces or Special Characters

Characters like `@`, `#`, `%`, and spaces are not allowed.

⚠️ Case-Sensitive

`itp`, `Itp`, and `iTp` are three different variables.

❌ No Reserved Keywords

Cannot use `if`, `for`, `while`, `True`, `class`, etc.

💡 **Best Practice:** Use meaningful names like `radius` or `total_score` instead of `a`, `b`, `c` for readable, maintainable code.

Case-Sensitivity in Action

File: 02cases.py

```
itp = 100
Itp = False
iTp = 5.782

print(itp, Itp, iTp)
```

Output:

```
100 False 5.782
```

Key Takeaway

Even though `itp`, `Itp`, and `iTp` look similar, Python treats them as **three completely separate variables**. Changing the case of even one letter creates a new variable.

⚠ Mixing up case is a common source of bugs. Always be consistent with your variable names.

2.3 Basic Data Types

Python automatically assigns a data type to a variable based on the value given. The four fundamental types are:

Type	Meaning	Example Values
`int`	Integer (whole number)	`10`, `-123`, `0`
`float`	Real number (decimal)	`3.14159`, `1.414`, `-0.5`
`str`	String (text)	`"Hello"`, `'Python'`
`bool`	Boolean (logical true/false)	`True`, `False`

Use the built-in `type()` function to check a variable's data type at any time.

Checking Data Types with type()

File: 02type.py

```
a = 10
b = 3.14
c = "Python"
d = True

print(type(a))
print(type(b))
print(type(c))
print(type(d))
```

Output

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

What This Shows

Each variable holds a different type. Python infers the type automatically – no explicit declaration is needed, unlike languages such as C or Java.

The Four Basic Data Types at a Glance

#

int

Whole numbers, positive or negative. Used for counting, indexing, and integer arithmetic.

10

float

Numbers with decimal points. Used for measurements, scientific values, and averages.

66

str

Sequences of characters enclosed in quotes. Used for names, messages, and text data.

bool

Only two values: `True` or `False`. Used for conditions and logical decisions.

2.4 Reassigning Variables

Variables can change their value – and even their type – during program execution. The new value simply replaces the old one.

File: 02reassign.py

```
x = 50
print(x)

x = 7
print(x)

x = "Baby"
print(x)
```

Output:

```
50
7
Baby
```

Dynamic Typing

Python is **dynamically typed** – a variable is not locked to one type. Here, `x` starts as an integer, becomes another integer, then becomes a string.

This flexibility is powerful but requires care to avoid unexpected type errors in calculations.

Multiple Assignment

Python allows assigning values to multiple variables simultaneously – both the same value and different values.

File: 02multiple.py

```
x = y = z = 28
print(x, y, z)

a, b, c = 3.56, 2, "Uncle"
print(a, b, c)
```

Output:

```
28 28 28
3.56 2 Uncle
```

Two Styles

Chained Assignment

`x = y = z = 28` assigns the same value to all three variables at once.

Tuple Unpacking

`a, b, c = 3.56, 2, "Uncle"` assigns different values in a single line.

2.5 Type Casting

Sometimes we need to convert data from one type to another. Python provides built-in functions for this:

int()

Converts to integer. **Truncates** (does not round) decimals. `int(3.99)` → `3`

float()

Converts to floating-point. `float(5)` → `5.0`

str()

Converts to string. `str(100)` → `"100"`

⚠ ⚠️ **Caution:** `int()` **truncates**, it does not round. `int(3.99)` gives `3`, not `4`. Use `round()` if rounding is needed: `round(3.99)` → `4`.

Type Casting in Practice

File: 02typecast.py

```
x = 3.54
print(int(x))      # truncate decimal
print(float(5))    # int to float
print(str(100))    # number to string
print(int("25") + 5) # string to int, then add
```

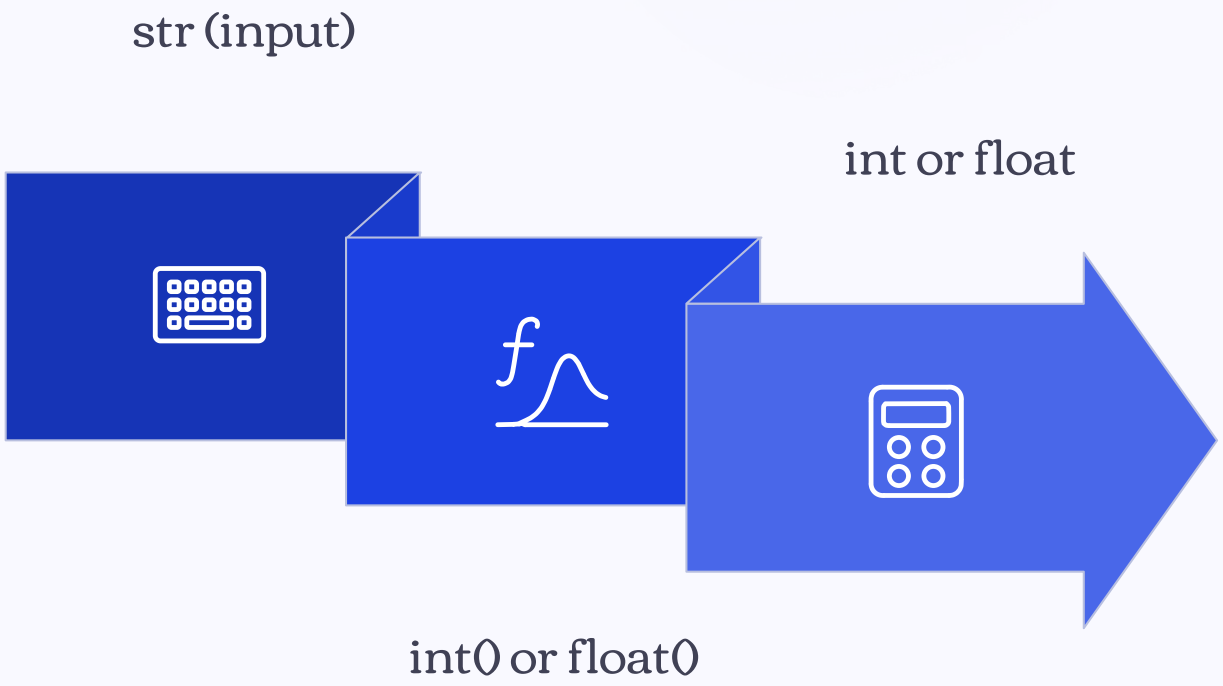
Output:

```
3
5.0
100
30
```

Why Type Casting Matters

The most common use case is converting the result of `input()` – which always returns a `str` – into a number before performing calculations.

Without casting, `"25" + 5` would raise a `TypeError`. With casting, `int("25") + 5` correctly gives `30`.



This conversion pipeline is essential whenever you receive numeric data from the user via `input()`, which always returns a string.

2.6 Receiving User Input with input()

How input() Works

The `input()` function displays a prompt message and waits for the user to type something and press Enter. The return value is **always of type**

The input() Function – Key Points

Always Returns a String

No matter what the user types – a number, a word, or a symbol – `input()` returns it as `str`.

Prompt is Optional but Recommended

The text inside the parentheses is displayed to guide the user. Without it, the program just waits silently.

Cast Before Calculating

Wrap with `int()` or `float()` immediately: `age = int(input("Age: "))`.

2.7 Arithmetic Operators

Python follows standard mathematical precedence: parentheses → exponentiation → multiplication/division → addition/subtraction.

Operator	Meaning	Example	Result
`+`	Addition	`7 + 3`	`10`
`-`	Subtraction	`7 - 3`	`4`
`*`	Multiplication	`7 * 3`	`21`
`/`	Division (float result)	`7 / 2`	`3.5`
`//`	Floor division	`7 // 2`	`3`
`%`	Modulo (remainder)	`7 % 2`	`1`
`**`	Exponentiation	`7 ** 2`	`49`

Arithmetic Operators – Code Example

File: 02operator.py

```
a = 17
b = 5

print("Quotient =", a // b)
print("Remainder =", a % b)
print("a squared =", a ** 2)
```

Output:

```
Quotient = 3
Remainder = 2
a squared = 289
```

Special Operators to Remember

// Floor Division

Divides and discards the decimal part.

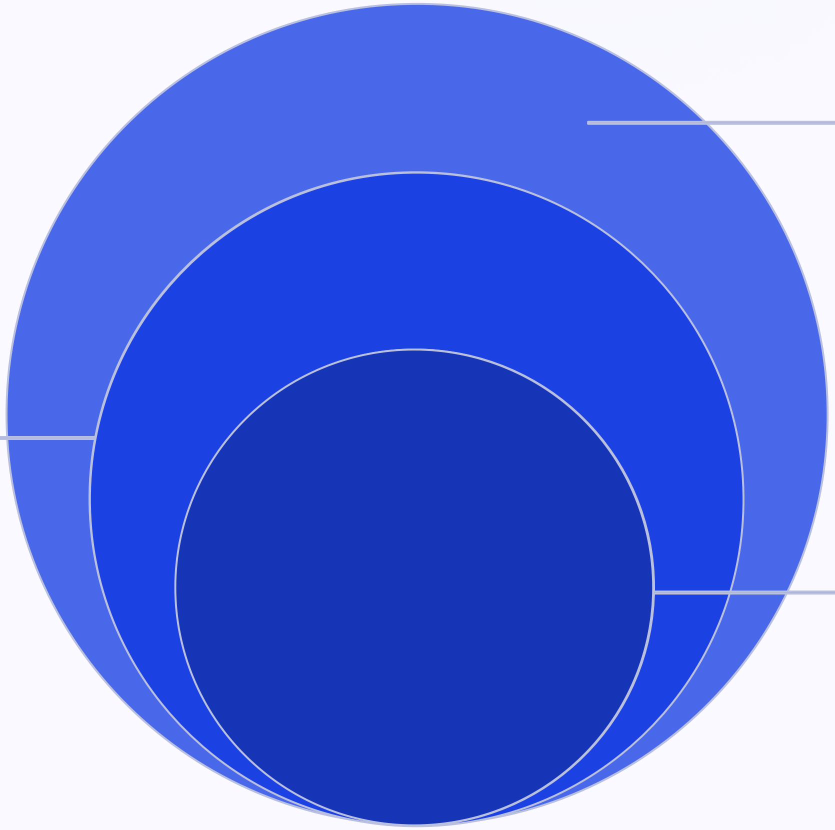
17 // 5 = 3

% Modulo

Returns the remainder.

17 % 5 = 2. Very useful for checking divisibility.

Operator Precedence



Mid & Low

*, /, //, % then +, - applied left-to-right

High Precedence

Exponentiation ** – power operations

Core

Parentheses () – override order

Always use parentheses to make your intended calculation order explicit and avoid ambiguity. For example, `(a + b) / 2` is clearer than `a + b / 2`, which divides only `b` by 2.

2.8 Formatting Output with f-strings

f-strings provide a clean, readable way to embed variable values directly inside a string. Add `f` before the opening quote, then place variable names inside `{ }`.

File: 02fstring.py

```
r = 5
pi = 3.14159
area = pi * r ** 2

print(f"Radius {r} units, "
      f"Circle area = {area:.2f} sq units")
```

Output:

```
Radius 5 units, Circle area = 78.54 sq units
```

Format Specifiers

{variable}

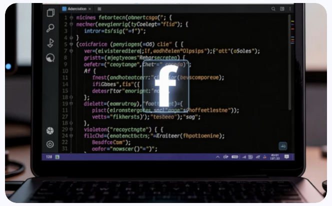
Inserts the variable's value as-is.

{variable:.2f}

Formats a float to exactly 2 decimal places.

f-strings are the recommended modern way to format output in Python 3.6+.

f-string Syntax at a Glance

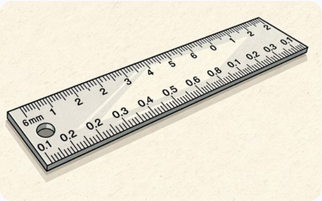


The f Prefix

Place `f` immediately before the opening quote: `f"..."` or `f'...'`

Curly Braces {}

Wrap any variable or expression inside `{ }` to embed it in the string.



Format Specifier :.2f

After a colon, specify format: `:.2f` means 2 decimal places for a float.

2.9 Applied Example: Average Score Calculator

This program combines all concepts from the chapter: `input()`, type casting, arithmetic, and f-string formatting.

File: 02average.py

```
s1 = float(input("Score 1: "))
s2 = float(input("Score 2: "))
s3 = float(input("Score 3: "))

avg = (s1 + s2 + s3) / 3
print(f"Average score = {avg:.2f}")
```

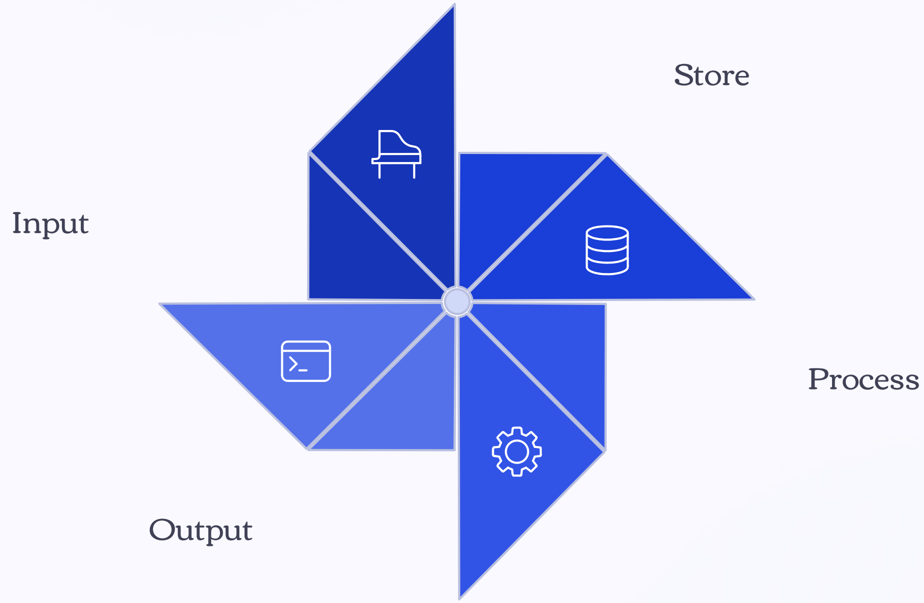
Sample Run

```
Score 1: 80
Score 2: 75
Score 3: 91
Average score = 82.00
```

Concepts Used

- `float(input())` – receive and cast input
- Arithmetic with parentheses for correct order
- `f"...{avg:.2f}"` – formatted output

Putting It All Together



Every program in this chapter follows the **IPO model** – Input, Process, Output – introduced in Chapter 1. Variables are the bridge that connects all three stages.

2.10 Chapter Summary

Variables

Store data with `=`. Names must follow Python rules. Case-sensitive.

Data Types

`int`, `float`, `str`, `bool`. Check with `type()`.

Type Casting

Convert with `int()`, `float()`, `str()`. Note: `int()` truncates, not rounds.

Input / Output

`input()` always returns `str`. Display with `print()` and f-strings.

Key Operators

`//` floor division and `%` modulo will be used frequently in later chapters.

Exercises – End of Chapter

1. **Variable Name Validity**
   State whether each name is valid and explain why: `2name`, `total_score`, `my-age`, `_value`, `class`

2. **Circle Calculator**
   Write a program that receives a circle's radius and displays the circumference ($2\pi r$) and area (πr^2) to 2 decimal places.

3. **Division & Remainder**
   Write a program that receives an integer and displays the quotient and remainder when divided by 7.

4. **Temperature Converter**
   Write a program that receives a temperature in Celsius and converts it to Fahrenheit using $F = C \times 9/5 + 32$.

5. **Conceptual Question**
   Explain why `int(input())` differs from `input()` alone.

EXERCISE ANSWERS

Exercise 1 – Variable Name Validity

Name	Valid?	Reason
`2name`	❌ Invalid	Starts with a digit
`total_score`	✅ Valid	Letters and underscore only
`my-age`	❌ Invalid	Contains a hyphen `-` (special character)
`_value`	✅ Valid	Underscore is allowed as first character
`class`	❌ Invalid	Reserved keyword in Python

EXERCISE 2 ANSWER

Circle Calculator

File: ex2_circle.py

```
r = float(input("Enter radius: "))
pi = 3.14159

print(f"Circumference = {2 * pi * r:.2f}")
print(f"Area = {pi * r ** 2:.2f}")
```

Sample Output

```
Enter radius: 7
Circumference = 43.98
Area = 153.94
```

Key Concepts Applied

- `float(input())` for decimal radius
- `**` for squaring the radius
- `:.2f` for 2 decimal places

EXERCISE 3 ANSWER

Division & Remainder by 7

File: ex3_division.py

```
n = int(input("Enter an integer: "))

print(f"Quotient = {n // 7}, "
      f"Remainder = {n % 7}")
```

Sample Output

```
Enter an integer: 50
Quotient = 7, Remainder = 1
```

Explanation

`50 // 7 = 7` because $7 \times 7 = 49$, and the floor of $50/7$ is 7.

`50 % 7 = 1` because $50 - 49 = 1$.

The `%` modulo operator is especially useful for checking if a number is even (`n % 2 == 0`) or divisible by any value.

EXERCISE 4 ANSWER

Temperature Converter: Celsius → Fahrenheit

File: ex4_temperature.py

```
c = float(input("Celsius: "))
f = c * 9 / 5 + 32

print(f"= {f:.2f} Fahrenheit")
```

Sample Output

```
Celsius: 37
= 98.60 Fahrenheit
```

The Formula

F = C × 9/5 + 32

Body temperature 37°C = 98.6°F. The formula uses standard arithmetic operators with Python's natural precedence – multiplication before addition.

Use float(input()) since temperatures can be decimal values.

EXERCISE 5 ANSWER

Why int(input()) Differs from input()

input() alone

Returns a `str` (string). Even if the user types `25`, the result is the text `"25"`, not the number 25.

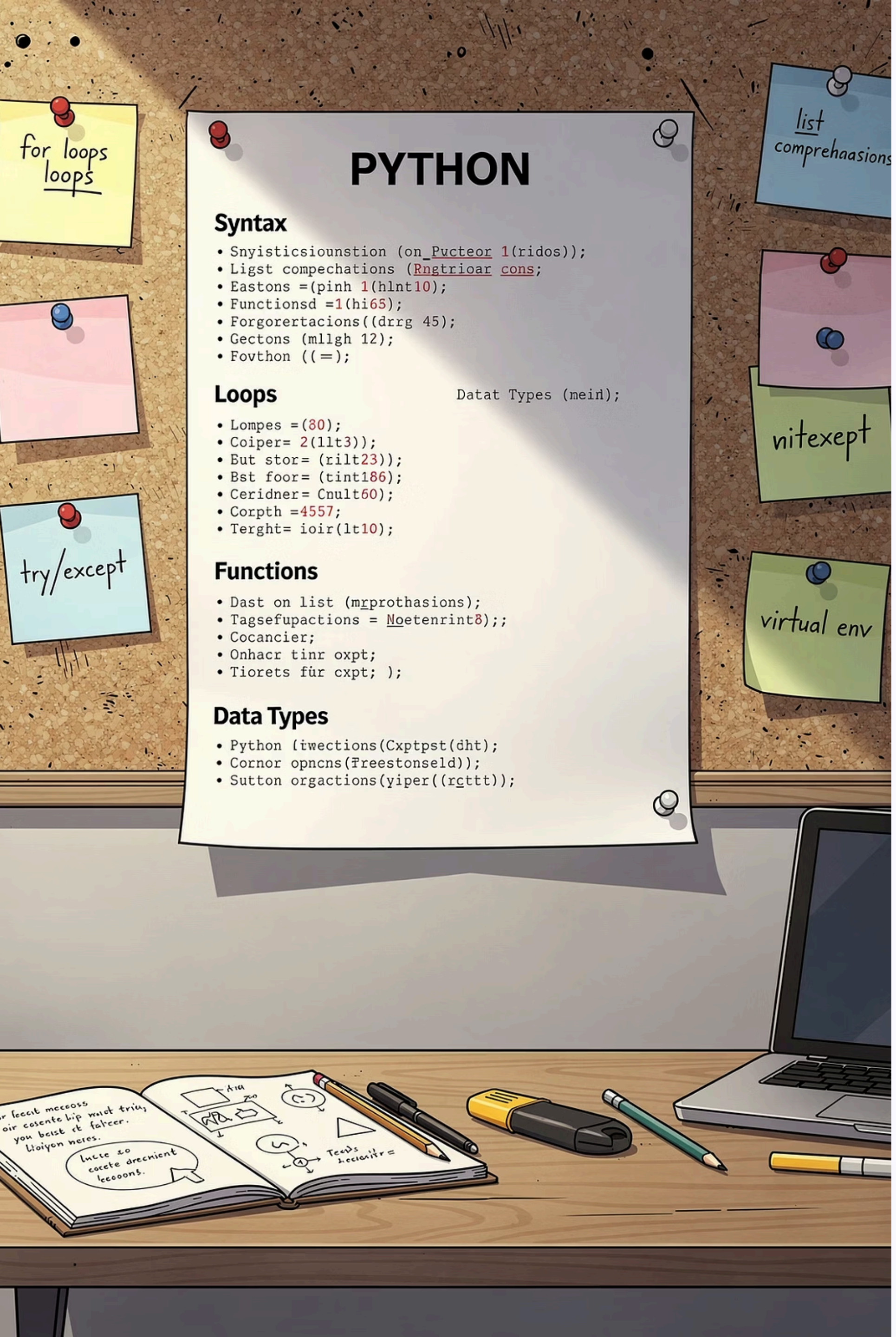
Attempting `"25" + 5` raises a `TypeError` – you cannot add a string and an integer.

int(input())

Wraps `input()` with `int()`, converting the returned string to an integer immediately.

Now `int("25") + 5 = 30` works correctly as numeric addition.

⚠ Always cast `input()` to the appropriate numeric type before performing any arithmetic operations.



Quick Reference: Variables & Types

Assignment

x = 10 | a, b = 1, 2 | x = y = z = 0

Check Type

type(x) → returns <class 'int'> etc.

Cast Types

int(x) | float(x) | str(x)

Input & Output

x = input("msg") | print(f"val={x:.2f}")

Quick Reference: Arithmetic Operators

+ Addition

`3 + 4 = 7`

- Subtraction

`7 - 3 = 4`

* Multiply

`3 * 4 = 12`

/ Divide

`7 / 2 = 3.5`

// Floor Div

`7 // 2 = 3`

% Modulo

`7 % 2 = 1`

** Power

`2 ** 8 = 256`

Common Mistakes to Avoid

Forgetting to Cast input()

Always use `int(input())` or `float(input())` when you need a number. Raw `input()` always returns a string.

Confusing = with ==

`=` is assignment (store a value). `==` is comparison (check equality). These are different operations.

Assuming int() Rounds

`int(3.99)` gives `3`, not `4`. Use `round()` when rounding is needed.

Using Reserved Keywords as Names

Names like `class`, `if`, `for`, `True` are reserved. Python will raise a `SyntaxError`.

int() vs round() – Important Distinction

int() – Truncation

Simply removes the decimal part. Does **not** round.

```
int(3.1)  → 3
int(3.5)  → 3
int(3.9)  → 3
int(-3.9) → -3
```

round() – Rounding

Rounds to the nearest integer (or specified decimal places).

```
round(3.1)    → 3
round(3.5)    → 4
round(3.9)    → 4
round(3.14159, 2) → 3.14
```

⚠ Use `int()` only when you intentionally want to discard the decimal. Use `round()` when mathematical rounding is required.

f-string Formatting Examples

f-strings support a variety of format specifiers beyond `:.2f`:

Format Specifier	Meaning	Example	Output
`{x}`	Default (as-is)	`f"{3.14159}"`	`3.14159`
`{x:.2f}`	2 decimal places	`f"{3.14159:.2f}"`	`3.14`
`{x:.0f}`	No decimal places	`f"{3.7:.0f}"`	`4`
`{x:10.2f}`	Width 10, 2 decimals	`f"{3.14:10.2f}"`	`      3.14`

ⓘ f-strings can also embed expressions directly: `f"{2 ** 10}"` outputs `1024`.

Complete Program Walkthrough

Let's trace through the average score program step by step:

1. ### Step 1: Input

   `float(input())` receives three scores as strings and converts them to floats.

2. ### Step 2: Store

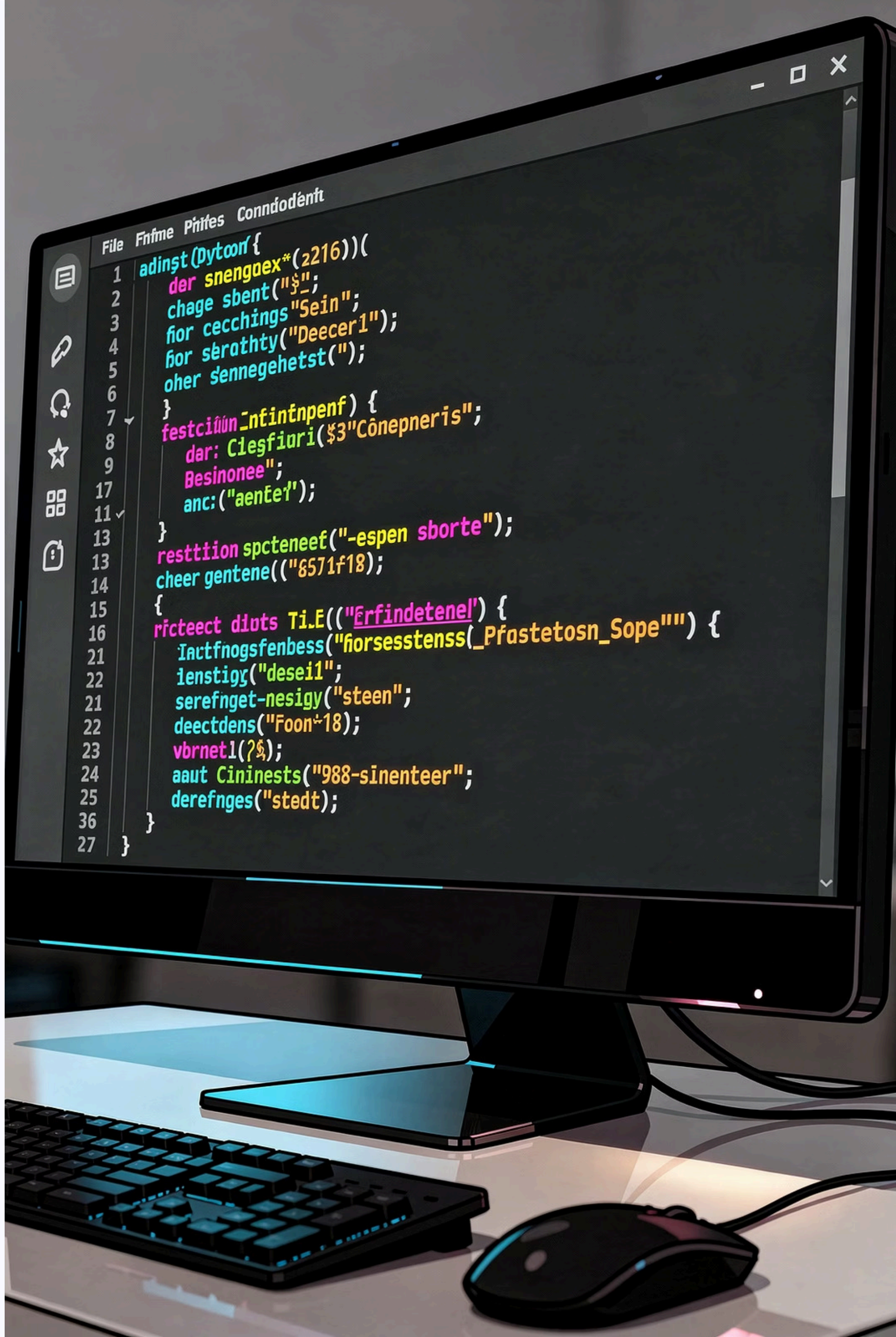
   Values are stored in variables `s1`, `s2`, `s3`.

3. ### Step 3: Process

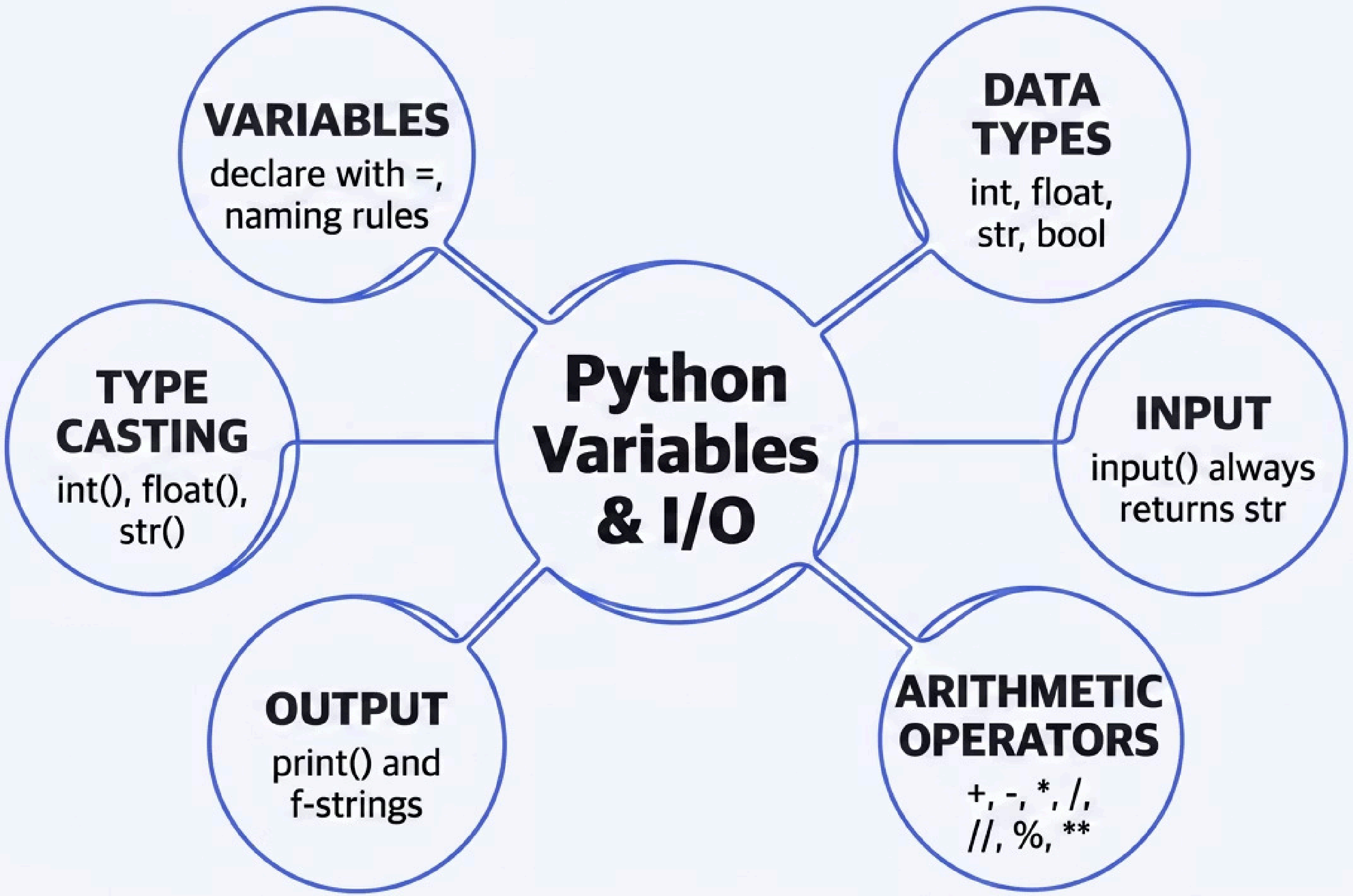
   `avg = (s1 + s2 + s3) / 3` computes the arithmetic mean.

4. ### Step 4: Output

   `print(f"...{avg:.2f}")` displays the result formatted to 2 decimal places.



Chapter 2 – Key Concepts Map



What's Coming Next

Chapter 2 Mastered ✅

- Variables and assignment
- Basic data types
- Type casting
- Input with `input()`
- Output with `print()` and f-strings
- Arithmetic operators

Chapter 3 Preview

The next chapter builds on these foundations to introduce **conditional statements** – using `if`, `elif`, and `else` to make programs that can make decisions based on variable values.

The `bool` type and comparison operators you've seen here will become central tools in Chapter 3.

> Make sure you are comfortable with all operators from this chapter before moving on.

Self-Check Questions

1

What does `input()` always return?

A value of type `str`, regardless of what the user types.

2

What is the result of `int(7.99)`?

`7` – because `int()` truncates, not rounds.

3

What does `10 % 3` evaluate to?

`1` – the remainder when 10 is divided by 3.

4

Are `Score` and `score` the same variable?

No – Python is case-sensitive. They are two different variables.

Chapter 2 Complete

> Variables are the memory of a program – without them, every computation would vanish the moment it was made.

📚 Review

Re-read sections on type casting and f-strings – these appear in every future program.

💻 Practice

Complete all 5 exercises. Run each program and verify your output matches the expected results.

🚀 Next Step

Proceed to Chapter 3: Conditional Statements – where programs start making decisions.